

Fundamentos de Retrieval-Augmented Generation (RAG)

Um Guia Introductório

Versão 1.0

Desenvolvido para utilização como base de conhecimento em sistemas RAG.

15 de julho de 2026

Sumário

1	Introdução	2
2	O que é um sistema RAG?	2
3	Base de Conhecimento	3
4	Chunking	3
4.1	Por que dividir documentos?	4
4.2	Chunk Overlap	4
5	Embeddings	5
5.1	Similaridade Semântica	5
5.2	Sentence Transformers	5
6	Banco Vetorial	6
6.1	Busca por Similaridade	6
6.2	Top-k	6
7	Retriever	7
7.1	Como o Retriever funciona?	7
7.2	Similarity Search	7
8	Prompt	8
8.1	Por que utilizar um Prompt?	8
9	Large Language Models	8
9.1	Modelos locais	9
9.2	Vantagens dos modelos locais	9
10	Integração entre Retriever e LLM	9
11	Arquitetura da Aplicação	10
12	Arquitetura em Camadas	11
13	Fluxo Completo da Pergunta	11
14	Organização do Projeto	12
14.1	Camada de Aplicação	12
15	Arquitetura da Observabilidade	13
16	LoggerManager	13
17	PerformanceMetrics	14
18	ExecutionContext	14
19	LLMMonitor	15
20	Integração entre os Componentes	15

1 Introdução

Nos últimos anos, os modelos de Inteligência Artificial baseados em *Large Language Models* (LLMs), como GPT, Llama, Mistral, Gemma e Qwen, revolucionaram a forma como interagimos com computadores. Esses modelos são capazes de responder perguntas, resumir documentos, gerar código, traduzir idiomas e realizar inúmeras outras tarefas relacionadas ao processamento de linguagem natural.

Apesar de extremamente poderosos, esses modelos apresentam uma limitação importante: eles não possuem acesso automático aos documentos específicos de um usuário ou de uma empresa. Em outras palavras, um LLM conhece apenas aquilo que esteve disponível durante seu treinamento e não sabe, por exemplo, responder perguntas sobre um manual interno, um contrato específico ou uma coleção particular de artigos científicos.

Foi justamente para resolver esse problema que surgiu o conceito de **Retrieval-Augmented Generation (RAG)**.

Em um sistema RAG, antes que o modelo de linguagem gere uma resposta, ocorre uma etapa intermediária responsável por recuperar documentos relevantes em uma base de conhecimento. Esses documentos são então incorporados ao contexto enviado ao LLM, permitindo que a resposta seja baseada em informações atualizadas e específicas do domínio da aplicação.

Essa arquitetura tornou-se uma das principais abordagens para desenvolvimento de assistentes inteligentes corporativos, sistemas de atendimento, buscadores semânticos e aplicações baseadas em Inteligência Artificial Generativa.

Observação

Um sistema RAG não modifica o conhecimento interno do modelo de linguagem. Em vez disso, ele fornece informações adicionais ao modelo durante a geração da resposta.

2 O que é um sistema RAG?

A sigla **RAG** significa *Retrieval-Augmented Generation*, ou, em tradução livre, *Geração Aumentada por Recuperação de Informação*.

A ideia central consiste em combinar duas tecnologias distintas:

- um mecanismo de recuperação de documentos (*Retriever*);
- um modelo de linguagem responsável pela geração da resposta (LLM).

O processo ocorre em duas etapas principais.

Primeiramente, o sistema recebe uma pergunta do usuário. Essa pergunta é convertida para uma representação vetorial e comparada com os vetores armazenados no banco de dados vetorial. Os documentos mais semelhantes são então recuperados.

Na segunda etapa, esses documentos são inseridos no *prompt* enviado ao modelo de linguagem. O LLM utiliza esse contexto adicional para produzir uma resposta baseada nas informações recuperadas.

Essa arquitetura reduz significativamente o problema conhecido como **alucinação**, no qual um modelo de linguagem inventa informações inexistentes.

A Figura 1 apresenta uma visão simplificada do funcionamento de um sistema RAG.

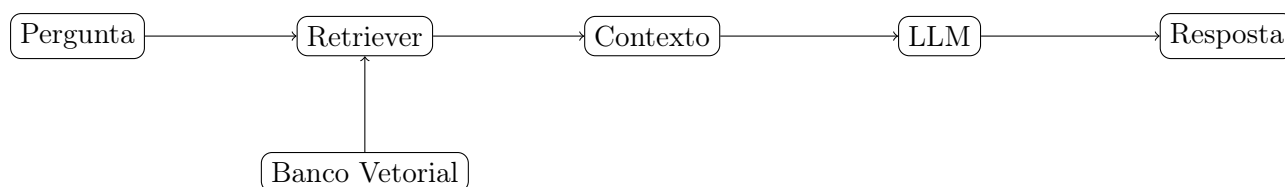


Figura 1: Fluxo simplificado de um sistema Retrieval-Augmented Generation.

3 Base de Conhecimento

Todo sistema RAG depende da existência de uma base de conhecimento.

Essa base consiste em um conjunto organizado de documentos que servirão como fonte de informação para responder às perguntas do usuário.

Os documentos podem possuir diversos formatos, como:

- arquivos PDF;
- documentos Word;
- páginas HTML;
- arquivos Markdown;
- planilhas;
- bancos de dados relacionais;
- APIs externas;
- páginas Wiki;
- documentação técnica;
- artigos científicos.

No projeto desenvolvido ao longo deste material, optou-se pela utilização de arquivos PDF como principal fonte de conhecimento. Essa escolha simplifica a construção da base documental e facilita sua atualização.

Sempre que novos documentos forem adicionados, removidos ou modificados, torna-se necessário reconstruir o banco vetorial para que os novos conteúdos sejam indexados.

Em termos práticos, isso significa executar novamente o processo de criação dos embeddings, responsável por converter cada trecho textual em sua representação vetorial.

Observação

A qualidade da base documental influencia diretamente a qualidade das respostas produzidas pelo sistema RAG. Um bom modelo de linguagem não consegue compensar uma base de documentos incompleta ou desatualizada.

4 Chunking

Um dos primeiros desafios encontrados na construção de um sistema RAG consiste em preparar os documentos para serem pesquisados posteriormente.

Embora seja possível armazenar um documento inteiro no banco vetorial, essa estratégia raramente produz bons resultados. Imagine, por exemplo, um livro com 300 páginas. Caso esse

livro fosse armazenado como um único documento, qualquer pergunta feita pelo usuário obrigaria o modelo a recuperar todo o conteúdo, aumentando significativamente o contexto enviado ao modelo de linguagem e reduzindo a precisão da recuperação.

Para contornar esse problema, os documentos são divididos em pequenos blocos de texto denominados **chunks**.

Cada chunk representa uma unidade relativamente independente de informação. Dessa forma, durante a recuperação, o sistema retorna apenas os trechos mais relevantes para responder à pergunta do usuário.

A Figura 2 ilustra esse processo.

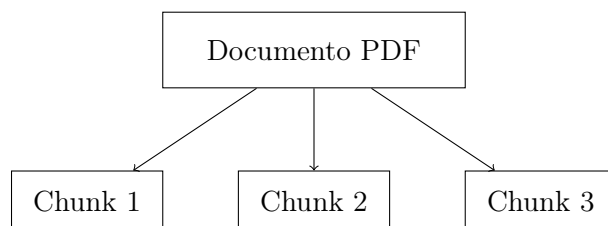


Figura 2: Divisão de um documento em múltiplos chunks.

4.1 Por que dividir documentos?

A divisão dos documentos produz diversas vantagens.

- reduz a quantidade de informação enviada ao modelo;
- melhora a precisão da recuperação;
- reduz o custo computacional;
- diminui a probabilidade de recuperar informações irrelevantes;
- melhora a qualidade das respostas.

Em geral, sistemas modernos utilizam chunks contendo entre 300 e 1000 palavras, dependendo da aplicação.

4.2 Chunk Overlap

Em muitos casos, uma informação importante encontra-se exatamente na fronteira entre dois chunks consecutivos.

Considere o seguinte exemplo.

Chunk 1

“O banco vetorial utiliza embeddings para representar documentos.”

Chunk 2

“...representar documentos em um espaço vetorial de alta dimensão.”

Caso não exista nenhuma sobreposição, parte da informação poderá ser perdida.

Por esse motivo, utiliza-se frequentemente uma estratégia denominada **chunk overlap**, na qual alguns caracteres ou palavras são repetidos entre chunks consecutivos.

Essa técnica preserva o contexto entre blocos adjacentes e melhora significativamente a recuperação semântica.

5 Embeddings

Depois que os documentos são divididos em chunks, surge um novo problema.

Computadores não compreendem diretamente o significado de palavras ou frases. Eles manipulam apenas números.

Assim, torna-se necessário converter cada trecho textual em uma representação numérica.

Essa representação é denominada **embedding**.

Um embedding consiste em um vetor numérico contendo centenas ou milhares de dimensões.

Por exemplo,

$$\mathbf{v} = (0.13, -0.41, 0.88, \dots, 0.27)$$

Esse vetor não possui interpretação direta para um ser humano.

Entretanto, modelos de aprendizado profundo conseguem organizar esses vetores de forma que textos semanticamente semelhantes ocupem posições próximas no espaço vetorial.

5.1 Similaridade Semântica

Considere as duas frases abaixo.

- O banco vetorial armazena embeddings.
- Os embeddings são armazenados em bancos vetoriais.

Embora as palavras estejam em ordens diferentes, ambas possuem praticamente o mesmo significado.

Modelos de embeddings geram vetores muito próximos para essas frases.

Por outro lado,

- A temperatura média da Lua é extremamente baixa.

produzirá um vetor distante dos anteriores.

Essa propriedade permite que a recuperação de documentos seja baseada no significado das frases, e não apenas nas palavras utilizadas.

5.2 Sentence Transformers

Neste projeto foi utilizado o modelo

`all-MiniLM-L6-v2`

da biblioteca Sentence Transformers.

Esse modelo foi treinado especificamente para produzir embeddings de alta qualidade para tarefas de recuperação de informação.

Embora existam modelos maiores, o MiniLM apresenta excelente equilíbrio entre velocidade, consumo de memória e qualidade dos embeddings, tornando-se bastante popular em aplicações RAG executadas localmente.

6 Banco Vetorial

Depois que todos os chunks foram convertidos em embeddings, eles precisam ser armazenados de maneira eficiente.

Essa função é desempenhada pelo **Banco Vetorial**.

Diferentemente de um banco de dados relacional tradicional, cujo objetivo principal consiste em armazenar tabelas, registros e relacionamentos, um banco vetorial foi projetado especificamente para realizar buscas por similaridade entre vetores de alta dimensão.

Neste projeto foi utilizado o **ChromaDB**.

Cada documento armazenado contém:

- o texto original;
- seu embedding;
- metadados;
- identificação da página;
- identificação do documento de origem.

6.1 Busca por Similaridade

Quando o usuário faz uma pergunta, ocorre exatamente o mesmo processo utilizado durante a indexação.

Primeiramente, a pergunta é convertida em embedding.

Em seguida, o banco vetorial calcula quais documentos possuem vetores mais próximos.

Quanto menor a distância entre dois embeddings, maior a similaridade semântica entre eles.

Esse processo é ilustrado na Figura 3.

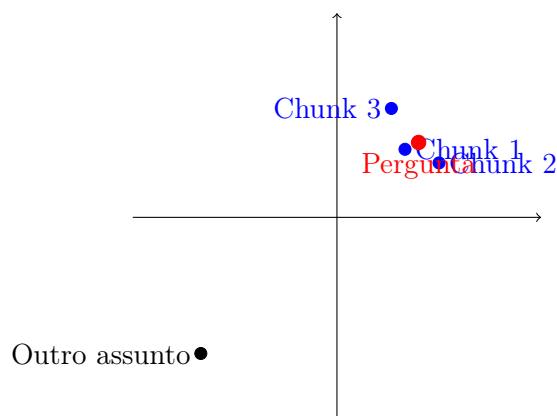


Figura 3: Busca baseada em proximidade entre embeddings.

6.2 Top-k

Após calcular a similaridade entre a pergunta e todos os embeddings armazenados, o banco vetorial retorna apenas os documentos mais relevantes.

Esse número é conhecido como **Top-k**.

Por exemplo,

$$k = 3$$

significa que apenas os três documentos mais semelhantes serão enviados ao modelo de linguagem.

Valores muito pequenos podem omitir informações importantes.

Valores muito grandes aumentam o contexto e podem reduzir a qualidade da resposta.

Observação

Chunking, embeddings e banco vetorial constituem o núcleo de qualquer sistema Retrieval-Augmented Generation. A qualidade dessas três etapas determina diretamente o desempenho do sistema durante a recuperação de documentos.

7 Retriever

Após a construção do banco vetorial, o sistema encontra-se preparado para responder perguntas dos usuários. Entretanto, antes que um modelo de linguagem possa gerar uma resposta, torna-se necessário selecionar quais documentos serão utilizados como contexto.

Essa tarefa é desempenhada pelo **Retriever**.

O Retriever é responsável por pesquisar o banco vetorial e identificar quais documentos apresentam maior similaridade semântica com a pergunta realizada pelo usuário.

Em outras palavras, o Retriever funciona como um mecanismo inteligente de busca, retornando apenas os trechos potencialmente relevantes para responder à pergunta.

7.1 Como o Retriever funciona?

Sempre que uma pergunta é enviada ao sistema, ocorre a seguinte sequência de operações:

1. a pergunta é recebida pelo sistema;
2. a pergunta é convertida em embedding;
3. o embedding é comparado com todos os vetores armazenados no banco vetorial;
4. calcula-se a similaridade entre a pergunta e cada documento;
5. selecionam-se os documentos mais relevantes (Top- k);
6. esses documentos são enviados ao modelo de linguagem.

A Figura 4 resume esse fluxo.



Figura 4: Etapas executadas pelo Retriever.

7.2 Similarity Search

A recuperação realizada pelo Retriever não é baseada apenas em palavras iguais.

Considere as perguntas abaixo.

- Como funciona um banco vetorial?
- O que faz o ChromaDB?

Embora essas perguntas utilizem palavras diferentes, elas tratam praticamente do mesmo assunto.

Como ambas possuem embeddings semelhantes, o banco vetorial retorna praticamente os mesmos documentos.

Essa característica diferencia um sistema RAG de mecanismos tradicionais baseados apenas em busca textual.

8 Prompt

Depois que os documentos relevantes são recuperados, eles precisam ser organizados antes de serem enviados ao modelo de linguagem.

Essa organização é realizada pelo **Prompt**.

Um prompt consiste no conjunto de instruções fornecidas ao modelo de linguagem durante a geração da resposta.

No projeto desenvolvido neste material foi utilizado um `ChatPromptTemplate` da biblioteca `LangChain`.

De maneira simplificada, o prompt possui a seguinte estrutura:

Exemplo de Prompt

```
Você é um assistente especializado em RAG.  
Utilize exclusivamente as informações abaixo.  
Contexto:  
{context}  
Pergunta:  
{input}
```

Observe que o campo `{context}` é preenchido automaticamente pelo Retriever com os documentos recuperados.

8.1 Por que utilizar um Prompt?

Modelos de linguagem são altamente sensíveis às instruções recebidas.

Um prompt bem elaborado pode:

- reduzir alucinações;
- melhorar a objetividade;
- restringir respostas ao contexto recuperado;
- definir o estilo da resposta;
- aumentar a precisão do sistema.

No presente projeto, o prompt instrui explicitamente o modelo a responder apenas com base na documentação recuperada.

9 Large Language Models

Depois que o prompt foi construído, inicia-se a última etapa do pipeline: a geração da resposta.

Essa etapa é executada por um **Large Language Model (LLM)**.

Um LLM consiste em uma rede neural treinada sobre enormes coleções de textos provenientes da internet, livros, artigos científicos e documentação técnica.

Durante o treinamento, o modelo aprende padrões estatísticos da linguagem, tornando-se capaz de gerar textos coerentes, responder perguntas e produzir código.

Entretanto, um LLM não consulta diretamente documentos externos durante sua utilização.

Por esse motivo, sistemas RAG recuperam informações adicionais antes da geração da resposta.

9.1 Modelos locais

Existem duas formas principais de utilizar modelos de linguagem.

A primeira consiste na utilização de APIs externas, como OpenAI, Anthropic ou Google Gemini.

Nessa abordagem, toda pergunta é enviada para servidores externos.

A segunda consiste na utilização de modelos locais.

Nesse caso, todo processamento ocorre no computador do próprio usuário.

Neste projeto foi utilizada essa segunda estratégia através do software **Ollama**.

O Ollama permite executar diversos modelos de linguagem localmente, como:

- Gemma;
- Llama;
- Mistral;
- DeepSeek;
- Qwen;
- Phi.

9.2 Vantagens dos modelos locais

Entre as principais vantagens destacam-se:

- maior privacidade;
- ausência de custos por requisição;
- funcionamento sem conexão permanente com a internet;
- possibilidade de utilizar documentos confidenciais;
- controle completo sobre o ambiente de execução.

Como desvantagem, modelos locais normalmente exigem maior capacidade computacional, especialmente memória RAM e processamento gráfico.

10 Integração entre Retriever e LLM

Uma das principais contribuições da biblioteca LangChain consiste em integrar automaticamente o Retriever e o modelo de linguagem.

No projeto desenvolvido neste material foi utilizada uma Retrieval Chain composta por duas etapas principais.

A primeira delas consiste em recuperar os documentos mais relevantes.

Em seguida, esses documentos são inseridos automaticamente no prompt enviado ao modelo.

Esse processo é ilustrado na Figura 5.



Figura 5: Fluxo entre Retriever, Prompt e LLM.

A biblioteca LangChain encapsula toda essa lógica por meio das funções

- `create_stuff_documents_chain()`;
- `create_retrieval_chain()`.

Essas funções permitem construir pipelines RAG completos com poucas linhas de código, mantendo uma arquitetura modular e facilmente extensível.

Observação

O Retriever não responde perguntas.
O modelo de linguagem também não pesquisa documentos.
O funcionamento correto de um sistema RAG depende da cooperação entre esses dois componentes: o Retriever seleciona os documentos mais relevantes e o LLM utiliza essas informações para construir a resposta final.

Arquitetura do Agente RAG

11 Arquitetura da Aplicação

Os conceitos apresentados nas seções anteriores descrevem o funcionamento geral de sistemas Retrieval-Augmented Generation.

Entretanto, transformar esses conceitos em uma aplicação robusta exige uma arquitetura de software organizada, modular e facilmente extensível.

Este capítulo apresenta a implementação desenvolvida neste projeto.

O objetivo foi construir um agente RAG local capaz de:

- responder perguntas sobre documentos PDF;
- utilizar modelos LLM executados localmente;
- armazenar memória conversacional;
- registrar métricas de desempenho;
- produzir logs estruturados;
- monitorar chamadas ao modelo;
- integrar-se ao MLflow;
- disponibilizar uma interface gráfica através do Streamlit.

A arquitetura foi organizada em módulos independentes, permitindo que novas funcionalidades sejam adicionadas sem modificar componentes já existentes.

12 Arquitetura em Camadas

Ao invés de concentrar toda a lógica em um único arquivo Python, a aplicação foi dividida em diversas camadas, cada uma responsável por uma tarefa específica.

Essa abordagem apresenta diversas vantagens:

- maior organização do código;
- facilidade de manutenção;
- reutilização de componentes;
- maior facilidade para testes;
- escalabilidade.

A Figura 6 apresenta a arquitetura geral da aplicação.

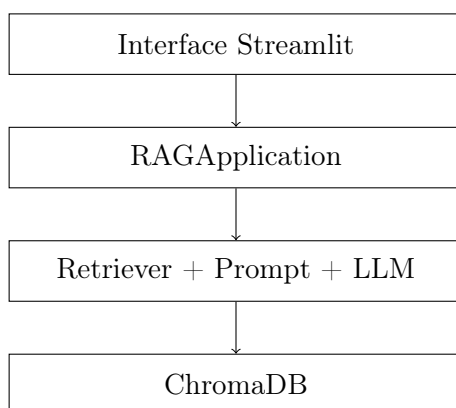


Figura 6: Arquitetura simplificada do agente RAG.

13 Fluxo Completo da Pergunta

Sempre que o usuário envia uma pergunta, diversas operações são executadas internamente.

O fluxo completo pode ser resumido nas seguintes etapas.

1. o usuário digita uma pergunta;
2. a interface Streamlit envia essa pergunta para a classe **RAGApplication**;
3. a memória conversacional recupera o histórico da sessão;
4. o Retriever consulta o banco vetorial;
5. os documentos mais relevantes são recuperados;
6. o Prompt é construído automaticamente;
7. o LLM gera a resposta;
8. a resposta é armazenada na memória;
9. métricas são coletadas;
10. logs são registrados;

11. os dados são enviados ao MLflow.

Observe que apenas uma pequena parte desse fluxo corresponde efetivamente à inferência realizada pelo modelo de linguagem.

Grande parte da aplicação dedica-se ao gerenciamento da execução, armazenamento de informações e monitoramento do sistema.

14 Organização do Projeto

Para tornar o código mais organizado, a aplicação foi dividida em diversos módulos.

A estrutura principal encontra-se organizada da seguinte forma:

```
src/  
  
application/  
memory/  
observability/  
llm/  
chain/  
prompts/  
retriever/  
vectorstore/
```

Cada diretório possui uma responsabilidade específica.

Essa separação reduz o acoplamento entre os componentes e facilita futuras expansões da aplicação.

14.1 Camada de Aplicação

A classe principal do sistema é denominada

`RAGApplication`.

Ela funciona como o ponto central de orquestração da aplicação.

Entre suas responsabilidades encontram-se:

- inicializar todos os componentes;
- executar consultas;
- controlar memória;
- registrar métricas;
- registrar logs;
- comunicar-se com o MLflow.

Na prática, toda interação do usuário passa por essa classe antes de alcançar o modelo de linguagem.

Observabilidade

Responder corretamente a uma pergunta representa apenas uma parte do funcionamento de um sistema RAG.

Em aplicações reais, é igualmente importante compreender como a resposta foi produzida, quanto tempo a execução levou, quais documentos foram recuperados, quantas chamadas ao modelo foram realizadas e quais erros eventualmente ocorreram durante a execução.

Esse conjunto de técnicas é denominado **observabilidade**.

A observabilidade permite monitorar continuamente o comportamento da aplicação, facilitando a identificação de gargalos de desempenho, erros de execução e oportunidades de otimização.

Neste projeto foi desenvolvida uma camada completa de observabilidade composta por cinco componentes principais.

- LoggerManager;
- PerformanceMetrics;
- ExecutionContext;
- LLMMonitor;
- MLflowManager.

15 Arquitetura da Observabilidade

A Figura 7 apresenta a arquitetura simplificada dessa camada.

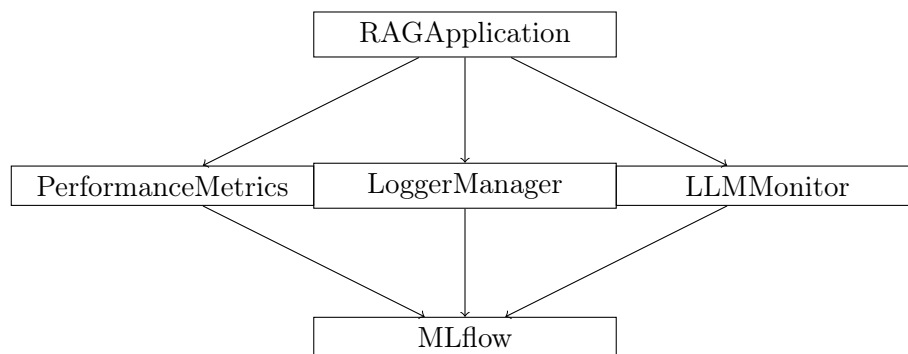


Figura 7: Arquitetura da camada de observabilidade.

16 LoggerManager

Todo evento relevante da aplicação é registrado automaticamente por meio da classe **LoggerManager**.

Ao invés de produzir apenas mensagens textuais no terminal, o projeto utiliza arquivos JSON estruturados.

Cada registro contém informações suficientes para reconstruir completamente uma execução da aplicação.

Entre os principais dados armazenados encontram-se:

- identificador da execução;
- data e horário;

- pergunta realizada;
- resposta produzida;
- duração da execução;
- status da operação;
- mensagens de erro (quando existentes).

O formato JSON facilita a integração futura com plataformas de monitoramento e ferramentas de análise de logs.

17 PerformanceMetrics

A classe `PerformanceMetrics` é responsável pela coleta automática das métricas de desempenho da aplicação.

Diferentemente de simples cronômetros, essa classe monitora simultaneamente diversos aspectos da execução.

As principais métricas registradas são:

- tempo total da requisição;
- tempo gasto pelo Retriever;
- tempo de geração do LLM;
- quantidade de documentos recuperados;
- tamanho do contexto;
- tamanho da pergunta;
- tamanho da resposta;
- estimativa do número de tokens.

Essas informações permitem identificar rapidamente gargalos de desempenho e avaliar o impacto de alterações futuras na arquitetura.

18 ExecutionContext

Cada pergunta enviada ao agente gera uma nova execução.

Para evitar perda de informações durante esse processo foi desenvolvida a classe `ExecutionContext`.

Essa classe centraliza todas as informações referentes à execução corrente.

Entre os principais dados armazenados encontram-se:

- identificador da execução;
- identificador da sessão;
- instante de início;
- instante de término;
- status da execução;

- informações de erro;
- interface utilizada (Terminal ou Streamlit).

A utilização dessa estrutura simplifica significativamente o fluxo de dados entre os diversos módulos da aplicação.

19 LLMMonitor

O modelo de linguagem representa o componente computacionalmente mais custoso de um sistema RAG.

Por esse motivo foi desenvolvido o módulo **LLMMonitor**.

Seu objetivo consiste em acompanhar continuamente a utilização do modelo.

Entre as métricas registradas destacam-se:

- número de chamadas ao modelo;
- número de chamadas bem-sucedidas;
- número de falhas;
- tempo acumulado de inferência;
- tempo médio por chamada.

Essas informações tornam-se particularmente úteis quando múltiplos usuários passam a utilizar simultaneamente a aplicação.

20 Integração entre os Componentes

Durante cada pergunta realizada pelo usuário, todas as classes de observabilidade trabalham de maneira integrada.

O fluxo ocorre da seguinte forma.

1. inicia-se uma nova execução;
2. o contexto da execução é criado;
3. o Logger registra o início;
4. o PerformanceMetrics inicia seus cronômetros;
5. o LLMMonitor acompanha a inferência;
6. a resposta é produzida;
7. métricas são calculadas;
8. logs são registrados;
9. os resultados são enviados ao MLflow.

A Figura 8 resume esse fluxo.

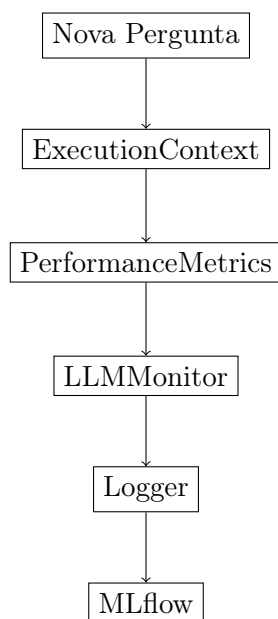


Figura 8: Fluxo da observabilidade durante uma execução.

Observação

Embora a camada de observabilidade não participe diretamente da geração das respostas, ela representa um componente essencial para aplicações de produção. A disponibilidade de logs estruturados, métricas detalhadas e monitoramento contínuo permite avaliar o comportamento do sistema ao longo do tempo, identificar gargalos de desempenho e facilitar o processo de manutenção e evolução da aplicação.